Termux Wayland Project — Practical Start

Vision UX importante

Le projet ne cherche PAS à créer un environnement virtuel Linux compliqué.

L'objectif est :

```
Transformer Android en lui ajoutant des capacités Linux desktop
sans root
sans configuration complexe
et avec une UX Android native.
```

Le projet doit rester :

- simple pour utilisateurs Android
- léger
- intégré au launcher Android
- compatible widgets/raccourcis Android
- proche d'un vrai système hybride Android + Linux.

Fonctionnalités UX prévues

Docks Android en widgets

L'application pourra créer :

- docks Linux-style sur écran d'accueil
- launchers flottants
- widgets 1×1
- rangées d'applications Linux
- mini taskbars Android
- menus applications Linux.

Raccourcis fichiers Android → Linux

L'utilisateur pourra créer :

- raccourcis vers fichiers
- ouverture directe de documents
- "Open with Linux app"
- associations fichiers Linux.

Exemples :

```
PDF → Okular Linux
ZIP → File Roller
HTML → Firefox Linux
TXT → Mousepad
```

Philosophie du projet

Le projet doit :

✅ étendre Android ✅ intégrer Linux naturellement ✅ garder l'UX Android ✅ éviter les desktops virtuels lourds ✅ éviter les configurations compliquées ✅ fonctionner sans root

et NON :

❌ remplacer Android ❌ créer une VM complète ❌ imposer un bureau Linux classique.

Goal

Create a single Android app:

```
com.termux.wayland
```

that:

- reuses existing Termux packages
- scans Linux desktop apps (.desktop)
- launches Linux apps individually
- uses Termux:X11 rendering backend initially
- later migrates to Wayland

Recommended Base

Start from the official Termux:X11 source code.

Reason:

- already solves Android ↔ native rendering
- already handles input
- already manages X11 socket communication
- already integrates SurfaceView/OpenGL
- much easier than extracting APK manually

Recommended repository:

https://github.com/termux/termux-x11

FIRST MVP

Before touching Wayland:

Build this:

```
Linux App Launcher for Android
```

Features:

1. Scan .desktop files
2. Show apps in RecyclerView
3. Launch Linux app through Termux
4. Display through Termux:X11

IGNORE:

- overlays
- multiple windows
- seamless mode
- GPU optimization
- widgets

for now.

Phase 1 Architecture

```
Android App
 ├── MainActivity
 ├── AppScanner
 ├── Launcher
 ├── X11SessionManager
 └── UI

Termux Environment
 ├── xterm
 ├── firefox
 └── xfce4-terminal
```

Important Decision

DO NOT rename everything immediately.

Keep Termux:X11 package names initially.

First:

- compile
- understand
- modify gradually

THEN:

- rename package
- restructure
- modularize

Otherwise debugging becomes painful.

STEP 1 — Build Original Termux:X11

Do NOT modify code yet.

First objective:

```
Compile and run original app.
```

Checklist:

- clone repo
- open in Android Studio
- install NDK
- install CMake
- build APK
- test on device

ONLY continue after this works.

STEP 2 — Understand Important Files

Android Java/Kotlin Side

Important folders:

```
app/src/main/java/com/termux/x11/
```

Likely important files:

```
MainActivity.java
LorieView.java
X11Activity.java
InputEventSender.java
```

Native Side

Important folders:

```
app/src/main/cpp/
```

Important concepts:

- EGL
- framebuffer
- input bridge
- X connection

STEP 3 — Add Linux App Scanner

Create:

```
LinuxApp.kt
LinuxAppScanner.kt
```

Goal:

Read:

```
/usr/share/applications
```

and parse:

```
Name=
Exec=
Icon=
```

LinuxApp.kt

```
package com.termux.wayland

data class LinuxApp(
    val name: String,
    val exec: String,
    val icon: String?
)
```

LinuxAppScanner.kt

Responsibilities:

- scan directories
- parse .desktop files
- return list<LinuxApp>

Initial scan paths:

```
/data/data/com.termux/files/usr/share/applications
```

Later:

```
proot distros
```

STEP 4 — Create Launcher UI

Use:

```
RecyclerView
```

Each item:

- icon
- app name
- click to launch

This becomes your Linux launcher.

STEP 5 — Launch Linux App

Simplest approach:

Use Termux RUN_COMMAND intent.

Example command:

```
export DISPLAY=:0
xfce4-terminal
```

or:

```
firefox
```

STEP 6 — Integrate Termux:X11 Session

Before launching app:

Start X11 server.

Then:

```
DISPLAY=:0
```

Then launch app.

IMPORTANT DEVELOPMENT STRATEGY

DO NOT rewrite whole codebase.

Instead:

Strategy

1. Keep original rendering backend
2. Add launcher layer
3. Add app scanning
4. Add session manager
5. Add seamless features later

This avoids getting lost in 1000+ files.

What We Will Do File-by-File

We should proceed in this order:

Phase A

Understand:

1. MainActivity
2. LorieView
3. native bridge
4. app startup flow

Phase B

Add:

1. LinuxApp.kt
2. LinuxAppScanner.kt
3. RecyclerView adapter
4. launcher activity

Phase C

Modify startup:

Instead of launching desktop:

launch selected Linux app.

VERY IMPORTANT

Do NOT try to understand all files.

Termux:X11 contains:

- rendering
- JNI
- EGL
- X11
- input
- Android lifecycle

You only need maybe:

```
20–40 important files
```

not 1000.

Next Practical Step

Your next task:

Clone and build Termux:X11 successfully.

Then send:

1. repo structure
2. MainActivity content
3. list of Java/Kotlin files

Then we continue file-by-file safely.